

Automatically Generating Programming Exercises with Open-Source LLMs: Integrating Lecture Slides and Learning Objectives

Marc Bessa Hoffmann

Institute of Software Systems
Hamburg University of Technology (TUHH)
Hamburg, Germany
`marc.bessa.hoffmann@tuhh.de`

Daniel Rashedi

Institute of Software Systems
Hamburg University of Technology (TUHH)
Hamburg, Germany
`daniel.rashedi@tuhh.de`

Sibylle Schupp

Institute of Software Systems
Hamburg University of Technology (TUHH)
Hamburg, Germany
`schupp@tuhh.de`

Automatic question generation using Large Language Models (LLMs) is an emerging area in educational technology that aims to reduce manual effort by instructors while generating pedagogically sound and contextually relevant exercises. However, generating questions that align with clearly defined Learning Objectives (LOs) while effectively incorporating contextual information from lecture slides remains a complex task that requires careful alignment between objectives and context. In programming education, existing approaches either rely on external contextual input or focus on learning objectives, but do not combine both. Current solutions are also limited to Multiple-Choice Questions (MCQs). This paper presents a framework for generating programming exercises for higher education. The framework combines LOs classified according to Bloom’s taxonomy with key concepts extracted from lecture slides. Using open-source LLMs, it produces diverse open-ended tasks, including implementation, explanation, and comparison exercises. We evaluated 60 exercises generated from predefined LOs and real-world lecture slides from a Haskell programming course. Each exercise was analyzed using an automated evaluation pipeline assessing grammar, readability, code validity, and semantic similarity to lecture content. Results show all exercises meet university-level quality standards for grammar and readability. Of the 60 exercises, 17 contained Haskell code snippets, of which 12 (71%) compiled without errors. Manual review confirmed strong alignment with predefined learning objectives and lecture content.

1 Introduction

LLMs are increasingly applied across a wide range of domains, including education [39]. In educational settings, they are used to help teachers generate instructional materials and exercises. This paper focuses on automatic question generation, where recent research has demonstrated promising results in tasks such as reading comprehension [42], computer programming [12], and solving mathematical problems [45]. We target programming exercises in the domain of functional programming using Haskell, a language commonly taught in university-level programming courses. While existing approaches often address introductory programming concepts, our work advances exercise generation to the academic level.

One of the core challenges in generating exercises is ensuring that they are meaningfully aligned with the intended learning objectives of a course. Learning objectives define the specific skills, knowledge, or competencies that students are expected to achieve by the end of a course. Ideally, these objectives

are structured according to established educational frameworks. A popular framework is Bloom’s taxonomy, which categorizes cognitive learning goals into six levels ranging from simple recall to higher-order skills [22]. At the same time, generated exercises should reflect the content of the lecture material taught in a course. Generating exercises that align with Bloom-based learning objectives while incorporating lecture content remains challenging, particularly when using general-purpose LLMs that are not optimized for educational purposes or the interpretation of structured teaching materials.

In the domain of programming education, existing solutions typically address only parts of the challenge. Some methods focus on generating questions based on educational materials [28]. Others attempt to guide the generation process using Bloom’s taxonomy, categorizing the output into cognitive levels [12, 34]. To our knowledge, no existing approach combines both contextual input from lecture slides and formalized learning objectives. Most research is also limited to Multiple-Choice Questions (MCQs). In programming, open-ended tasks such as implementation, explanation, or comparison are often more meaningful and authentic. Most existing approaches rely on commercial models like GPT-3 or GPT-4, which limits transparency and reproducibility.

To address these limitations, we present an exercise generation framework that leverages open-source LLMs. A core design goal of our framework is accessibility: the pipeline should be usable by educators without specialized infrastructure, enabling deployment on standard consumer hardware. Specifically, we employ Qwen2.5-Coder, an open-source model that has shown strong performance on code understanding and generation benchmarks, making it well suited for programming-related tasks [19]. Our framework combines two sources of guidance: Learning Objectives (LOs) categorized by Bloom’s taxonomy and lecture slides that contain the actual teaching material of a course. The framework takes as input an LO, a topic, and a set of lecture slides. The LO is automatically classified according to Bloom’s taxonomy using a rule-based verb matching approach, where specific action verbs (e.g., define, explain, evaluate) are mapped to corresponding Bloom levels. Based on the assigned Bloom level, the framework selects predefined example exercises that reflect the corresponding cognitive demands and serve as structural guidance in the prompt. A vector-based similarity search is then performed between the LO and the lecture slides to retrieve the most relevant content. This mechanism follows the Retrieval-Augmented Generation (RAG) approach, where external context grounds the model output [24]. We use the embedding model all-MiniLM-L6-v2 for this purpose. The combined input enables the generation of programming exercises that are both grounded in course content and aligned with the intended learning outcomes. Depending on the cognitive Bloom level, the framework can generate a variety of task types including syntax recall, implementation tasks, conceptual explanation, and comparative evaluation.

To assess the quality of the generated exercises, we applied a fully automated five-stage evaluation pipeline. This pipeline verified (1) grammatical correctness using rule-based grammar analysis, (2) readability based on Flesch Reading Ease, (3) compilability of embedded Haskell code using the Glasgow Haskell Compiler (GHC), (4) semantic similarity between exercises and the related lecture slides using sentence embeddings, and (5) alignment with the intended Bloom level through LLM-based classification. The results show that all exercises were grammatically correct, with only minor false positives caused by embedded code elements. Readability scores were appropriate for university-level material. Of the 17 exercises containing Haskell code, 12 (71%) compiled successfully. Manual review of selected examples confirmed strong alignment with both the intended learning objectives and the course content.

This paper is structured as follows: First, we provide essential background information by introducing fundamental concepts related to LLMs, embeddings, prompt engineering, and Bloom’s taxonomy. After that, we review related work in the field. Building upon this foundation, we then describe our own approach for generating programming exercises. Finally, we present and discuss the evaluation results. This work is based on a research project conducted at TU Hamburg [18].

2 Background

This section introduces the core concepts underlying our work. We first explain LLMs and prompt engineering, which form the technical foundation for exercise generation. We then discuss embeddings, which enable semantic retrieval of relevant lecture content. Finally, we introduce Bloom’s taxonomy as the educational framework used to structure and classify learning objectives.

2.1 Large Language Models

LLMs are based on the Transformer architecture introduced by Vaswani et al. [38], which overcomes the limitations of earlier models such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory Networks (LSTMs). Through the self-attention mechanism, Transformers capture relationships between all tokens in a sequence simultaneously, enabling them to model long-range dependencies and contextual meaning.

Modern LLMs such as BERT [11] and GPT-3 [4] are trained on large-scale datasets containing text from books, websites, programming code, and other human-generated content [36]. During training, these models learn statistical and semantic relationships between words and concepts, enabling them to understand, reason, and generate human-like text across various domains. This capability makes them a powerful foundation for educational applications, where generating coherent explanations, exercises, or feedback requires both linguistic and conceptual understanding. For the purpose of this work, models trained on programming-related data such as Qwen2.5-Coder are particularly relevant, as they can not only generate syntactically valid code for the exercises but also comprehend and reason about existing code snippets that may appear in the lecture slides.

To harness these capabilities, the model’s behavior must be guided through well-designed input instructions. This process, known as *prompt engineering*, determines how an LLM interprets tasks and produces its outputs.

Prompt Engineering Clear and precise instructions are essential, since vague prompts often lead to generic or irrelevant responses. Role prompting, for example, allows the model to act as a specialized assistant for programming exercise generation. Another approach is to include examples within the prompt to help the model reproduce the desired structure and complexity. Depending on the number of examples provided, this is referred to as zero-shot (no examples), one-shot (a single example), or few-shot prompting (several examples) [7]. More advanced techniques improve reasoning and consistency, such as Chain-of-Thought prompting [41] or self-consistency [40]. In our work, we apply role prompting and few-shot prompting to construct prompts that generate structured programming exercises.

2.2 Embeddings

Embeddings provide the mathematical foundation for representing and processing natural language in machine learning models. The core idea is to transform words, sentences, or even entire documents into numerical vectors. These vector representations capture the semantic meaning and enable reasoning using mathematical operations. Figure 1 shows example embeddings for a learning objective and two slide sentences.

To quantify the semantic relationship between these vectors, similarity metrics such as cosine similarity are applied. These metrics measure how closely two embeddings align in direction, where a higher

$$\text{LO} = \begin{bmatrix} 0.24 \\ 0.51 \\ -0.15 \\ \vdots \\ 0.11 \end{bmatrix}, \quad \text{Slide A} = \begin{bmatrix} 0.21 \\ 0.53 \\ -0.13 \\ \vdots \\ 0.09 \end{bmatrix}, \quad \text{Slide B} = \begin{bmatrix} 0.11 \\ 0.34 \\ 0.18 \\ \vdots \\ 0.02 \end{bmatrix}.$$

Figure 1: Example embeddings of a learning objective (LO) and two slide sentences.

value indicates stronger semantic alignment. A possible 2D projection of these vectors is shown in Figure 2, illustrating that Slide A is semantically closer to the learning objective than Slide B.

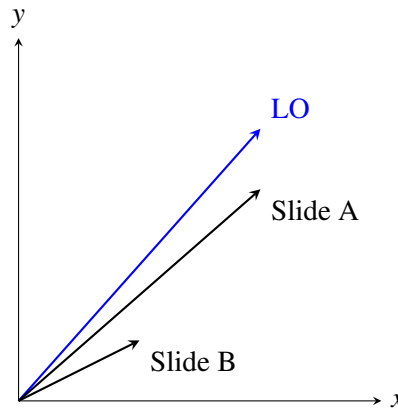


Figure 2: 2D projection of embedded vectors illustrating the semantic proximity between a learning objective (LO) and two lecture slides. Slide A lies closer to the LO in the embedding space than Slide B and is therefore retrieved as more semantically relevant.

In our approach, we embed learning objectives and lecture slides to compute semantic similarity and identify the slides that are most semantically relevant for each objective.

2.3 Bloom's Taxonomy

When generating educational exercises, it is essential to consider that different types of exercises serve different purposes. Some exercises focus on recalling facts, while others require critical thinking or problem solving. To address these differences, a structured framework is necessary. Bloom's taxonomy is a classification system for learning objectives developed by Benjamin Bloom et al. in 1956 [22]. The taxonomy was revised in 2001, updating some of the original terms and refining the categorization of learning skills [1]. It defines six different levels that categorize the cognitive processes involved in learning. The six learning levels consist of the following.

- **Remember** – Retrieving, recognizing, and recalling relevant knowledge from long-term memory.
- **Understand** – Constructing meaning from oral, written, and graphic messages through interpreting, exemplifying, classifying, summarizing, inferring, comparing, and explaining.

- **Apply** – Carrying out or using a procedure for executing or implementing.
- **Analyze** – Breaking material into constituent parts and determining how the parts relate to one another and to an overall structure or purpose through differentiating, organizing, and attributing.
- **Evaluate** – Making judgments based on criteria and standards through checking and critiquing.
- **Create** – Putting elements together to form a coherent or functional whole; reorganizing elements into a new pattern or structure through generating, planning, or producing.

In this work, we use Bloom’s taxonomy to classify learning objectives and to generate programming exercises that target the corresponding Bloom level.

3 Related Work

Early approaches to automatic question generation were predominantly rule-based or relied on syntactic and semantic transformation techniques [23]. In recent years, research has increasingly shifted toward leveraging LLMs. This section reviews related work and highlights how our approach differs from existing methods.

Only two approaches directly address question generation in the context of programming education. Doughty et al. [12] compare GPT-4-generated multiple-choice questions with those authored by humans in programming education and employ a fine-tuned BERT model to classify learning objectives according to Bloom levels. EvalQuiz [28] explores the automatic generation of self-assessment quizzes from lecture slides using GPT-4. Both approaches demonstrate the growing potential of LLMs for supporting programming education, yet they remain limited to multiple-choice question formats.

Beyond programming-specific work, several studies have investigated the use of Bloom’s taxonomy for guiding question generation across broader educational domains. Scaria et al. [34] applied Bloom’s taxonomy to generate educational questions across multiple domains and cognitive levels, using both open-source and GPT-based models. In our work, we also incorporate Bloom’s taxonomy to address the cognitive dimension of generated programming exercises. Instead of using machine learning-based classifiers, we apply a keyword-matching approach to associate learning objectives with appropriate Bloom levels.

Another important aspect is the use of additional context to improve the quality and relevance of generated questions. RAG, first introduced by Lewis et al. [24], retrieves external context to ground the model’s output and reduce hallucinations. In the educational domain, Maity et al. [25] successfully applied RAG to generate questions from course materials, demonstrating that contextually grounded question generation can lead to more accurate and pedagogically relevant results. In our work, we incorporate RAG into our pipeline by using lecture slides as a contextual basis for exercise generation.

Although context and cognitive level are essential factors, the format of the question also shapes how learners interact with and reflect on the material. Despite growing interest in LLM-based question generation, most existing research in education has focused primarily on MCQs [29, 12]. In contrast, our approach is not limited to MCQs but supports the generation of exercises across various formats, including open-ended and coding-related tasks. To guide the model in generating such diverse exercise types, we provide example tasks that resemble academic exercises.

Our approach extends previous work by combining the following aspects:

- going beyond MCQs to generate diverse open-ended programming exercises,
- incorporating lecture slides as structured educational context,

- integrating learning objectives and applying Bloom’s taxonomy to classify the cognitive level, and
- relying on open-source models to ensure transparency.

While related work addresses individual aspects, no existing study combines these components into a unified pipeline for generating programming exercises.¹

4 Approach

This section outlines our approach to generate programming exercises that are aligned with two key elements: learning objectives categorized by Bloom’s taxonomy (BT) and the specific content of the lecture slides. We present a structured pipeline that transforms pedagogical input into targeted programming exercises using LLMs. An overview of the entire process is illustrated in Figure 3.

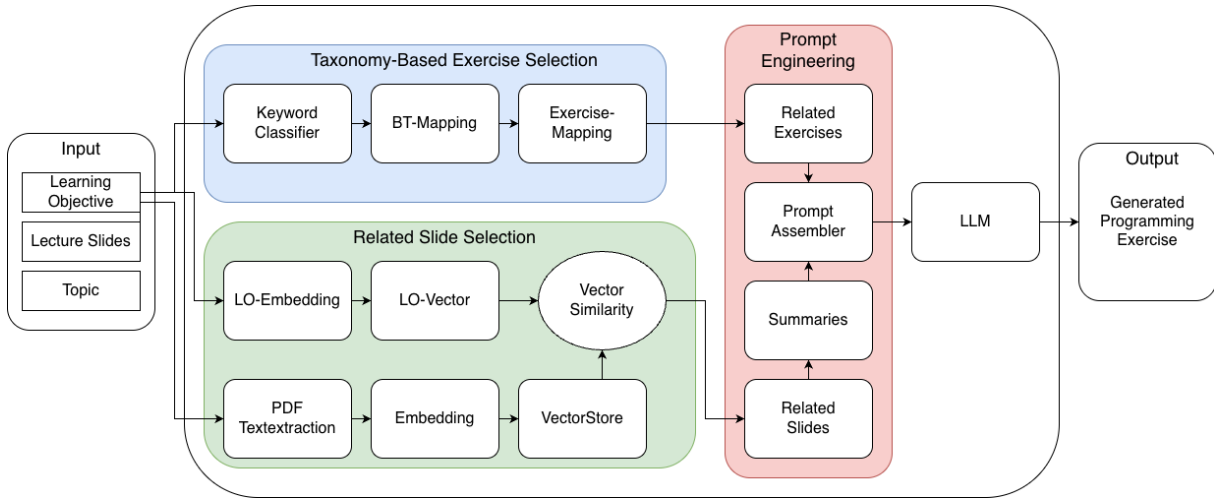


Figure 3: Three-stage exercise generation pipeline: (1) semantic retrieval of lecture slides relevant to a given learning objective, (2) structured prompt construction combining the LO with the retrieved slides, and (3) exercise generation by an open-source LLM.

The pipeline requires three inputs from instructors: a *topic* that defines the thematic scope (e.g., a subtopic like conditionals or a programming language like Haskell), a *learning objective* that describes the intended educational outcome, and *lecture slides* that provide contextual foundation regarding notation, terminology, and concepts used in the course. Based on this input, the generation process consists of three main stages:

- **Taxonomy-Based Exercise Selection:** Classifies the learning objective into one of Bloom’s cognitive levels using keyword matching. For each level, predefined example exercises illustrate how programming exercises should look at the corresponding level of complexity.
- **Related Slide Selection:** Retrieves the most relevant lecture slides through a retrieval-augmented approach. The selected slides are summarized using an LLM to extract key concepts and code examples.

¹This paper is based on a research project conducted at TU Hamburg [18].

- **Prompt Engineering:** Combines the example exercises and the summarized slides into a structured prompt that serves as input for generating the final programming exercise.

The following sections describe each stage of the pipeline in detail and explain their role in the overall exercise generation process. As illustrated in Figure 3, the stages are presented from left to right: the process begins with taxonomy-based exercise selection and related slide retrieval, which together perform the preprocessing necessary for the final stage of prompt engineering.

4.1 Taxonomy-Based Exercise Selection

The taxonomy-based exercise selection process consists of two stages. In the first stage, the learning objective is classified according to Bloom’s taxonomy using keyword matching. In the second stage, the resulting Bloom level is mapped to the corresponding exercise sheets. Each exercise sheet defines how a coding exercise should be structured for the designated learning level.

Bloom Level Classification To generate an exercise that targets an appropriate cognitive level, the learning objective must first be classified according to Bloom’s taxonomy. The core idea is to analyze the verbs contained in the objective and match them against taxonomy keywords. The classification involves three basic steps: tokenizing the sentence, reducing the verbs to their base form, and comparing them with the Bloom keyword lists. A possible implementation is shown in Algorithm 1.

Algorithm 1 Classification of Learning Objective by Bloom Level

```

1: function CLASSIFYBLOOMLEVEL(learningObjective)
2:   Input: learningObjective (string)
3:   Output: matchedLevels (set of Bloom levels)
4:   tokens ← TOKENIZE(learningObjective)
5:   tagged ← POSTAG(tokens)
6:   verbs ← EXTRACTVERBS(tagged)
7:   stems ← STEM(verbs)
8:   matchedLevels ← {}
9:   for all level in BloomLevels do
10:    if ANYMATCH(stems, bloomVerbs[level]) then
11:      MATCHEDLEVELS.ADD(level)
12:    end if
13:  end for
14:  return matchedLevels
15: end function

```

The output is a set of matched Bloom levels, since one learning objective may contain verbs from different categories. In our current implementation, the first detected level is selected as the representative one. However, alternative approaches could involve instructors manually determining the most appropriate Bloom level when several levels are plausible or no match could be found.

Exercise Selection Once the Bloom level is identified, the next step is to select exercise types that reflect the corresponding cognitive complexity. To support this, we have defined example exercises for each Bloom level that serve as few-shot prompts and guide the model toward the expected structure and difficulty.

The selection of appropriate exercise types is based on the cognitive demands of each Bloom level. For programming education, we identified task formats that align with the intended learning depth:

- **Remember:** Recalling facts, e.g., naming keywords or listing built-in functions.
- **Understand:** Explaining concepts, e.g., describing function behavior or identifying type errors.
- **Apply:** Using knowledge in practice, e.g., writing functions or rewriting code with recursion.
- **Analyze:** Breaking down or comparing implementations, e.g., explaining run-time behavior.
- **Evaluate:** Judging correctness or performance, e.g., comparing two implementations.
- **Create:** Developing new solutions, e.g., designing data structures or full implementations.

For each level, we provide three concrete example exercises in JSON format that illustrate the expected structure, task formulation, and cognitive depth. These examples are included in the prompt to guide the model through few-shot learning. Together with the lecture slide content (described in the next subsection), they form the contextual input for exercise generation.

4.2 Related Slide Selection

The second component of the pipeline retrieves lecture slides relevant to a given learning objective. These slides provide contextual input on terminology, notation, and code examples for the final prompt. Figure 4 illustrates this retrieval process. This structure follows the general idea of RAG, where external context is retrieved to ground the model output.

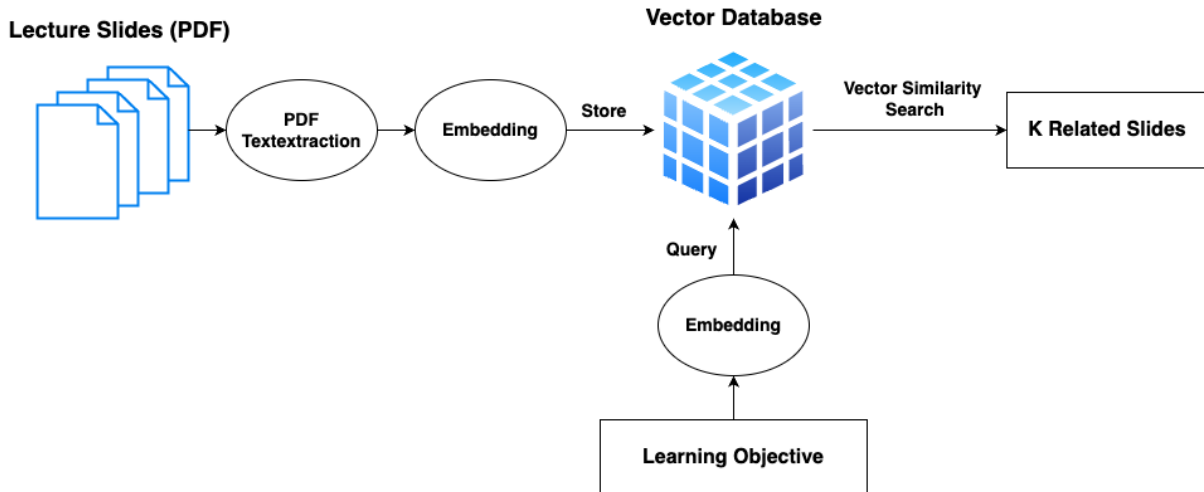


Figure 4: Semantic Retrieval Pipeline for Identifying Relevant Lecture Slides.

The process consists of three steps. First, we extract the textual content of each PDF slide using standard parsing libraries (e.g., PyMuPDF). Second, slide texts are embedded into a semantic vector space using open-source sentence transformers (e.g., all-MiniLM-L6-v2) and stored in a FAISS vector database [13]. Finally, the learning objective is embedded in the same space and used to retrieve the most relevant slides via a top- k similarity search, where k denotes the number of nearest-neighbor slides returned. A possible implementation of this retrieval process is shown in Algorithm 2.

The output of this stage is a set of slide texts most relevant to the given learning objective. Before being passed to the prompt construction, these slides are summarized by an LLM to extract key points, with

Algorithm 2 Related Slide Selection Based on Learning Objective

```

1: function SELECTRELATEDSLIDES(lectureSlides, learningObjective)
2:   slideTexts  $\leftarrow$  EXTRACTTEXTFROMPDF(lectureSlides)
3:   for all slideText in slideTexts do
4:     vector  $\leftarrow$  EMBED(slideText)
5:     STOREINVECTORDB(vector)
6:   end for
7:   queryVector  $\leftarrow$  EMBED(learningObjective)
8:   relatedSlides  $\leftarrow$  VECTORSIMILARITYSEARCH(queryVector,  $k = 3$ )
9:   return relatedSlides
10: end function

```

a focus on technical content and code snippets. The resulting summaries are stored as structured JSON objects that serve as contextual input for exercise generation.

4.3 Prompt Engineering

This section describes the implementation of the final component of the pipeline. The results of the previous stages provide the foundational input for the prompt engineering phase, namely exercise example sheets that illustrate programming tasks appropriate for the given Bloom level and summaries of lecture slides identified as relevant to the learning objective. Figure 5 shows the final version of the structured prompt used to generate programming exercises. The prompt design is based on best practices in prompt engineering and follows the structure proposed by Doughty et al. [29]. It is composed of five core components.

First, the framework is explicitly instructed to act as a learning engineer support bot. This role-based instruction encourages the language model to adopt a pedagogically grounded perspective when generating exercises. Second, to guide the cognitive alignment, the six levels of Bloom’s taxonomy are included in the prompt. By defining these levels, the model is made aware of the desired complexity of the generated output. Third, the contextual input consists of the exercise topic, the associated learning objective, and summaries of the relevant lecture slides. The topic helps narrow the focus of the exercise, the learning objective describes the intended skill or knowledge outcome, and the slide summaries reflect terminology and concepts taught in the course. Fourth, to support the generation of well-structured exercises, three example exercises are included in the prompt. These examples apply the principle of few-shot prompting and help the model understand how the output should be formatted. Finally, the prompt defines a fixed output format: a JSON object that contains the exercise type, the exercise text, and optional subtasks if applicable. This structure encourages the model to produce complete and potentially multi-step exercises. Once the prompt is fully constructed, it is passed to the language model, which then generates the final programming exercise.

You are a learning engineer support bot tasked with creating high-quality, self-contained programming exercises.

Role Prompting

Bloom's Taxonomy

The programming exercise must be well aligned with the learning objective it is intended to assess. This means it must target the correct cognitive complexity as defined by Bloom's Taxonomy. Below are the six levels of Bloom's Taxonomy, each corresponding to a specific type of thinking skill:

Remember – Retrieve or recognize facts, basic concepts, or syntax.

Understand – Explain ideas or concepts; interpret and summarize code or behavior.

Apply – Use knowledge in new situations, such as implementing a known algorithm or completing a function.

Analyze – Break down code to understand structure or compare different implementations.

Evaluate – Make informed judgments about code correctness, performance, or quality.

Create – Produce new code, design a system, or restructure an existing solution creatively.

Contextual Input

The **topic** of the exercise should lie within the field of: **Higher Order Functions in Haskell**

The exercise must be aligned with the following **learning objective**: **Students should be able to apply higher-order functions like map and filter to process lists**

Learning objectives define what students should be able to do after completing the question.

Use the following summaries of lecture slides as internal context. They are taken from real course materials and provide relevant terminology, code patterns, and notations.

Lecture Slide Summaries:

- map applies a function to each element in a list.
- filter selects elements based on a condition

Exercise Type & Example

Target Bloom Level: Apply

Apply-level exercise types ask students to use their knowledge in new but similar situations. These typically involve completing or modifying code, implementing a small function, or applying a known algorithm to a concrete example.

Example Assignments:

- Provide a Haskell expression that returns a list of all integers between 1 and 99 that are divisible by 3.
- Given two lists [4, 1] and [3, 2], provide a Haskell expression that returns the list [1, 2, 3, 4] using head, tail, !!, ++, take, drop, reverse, or init.
- Express the list comprehension $[x * x \mid x \leftarrow [1..], \text{even } x]$ using map and filter.

Output Format

```

{{
  "exercise_type": "<Short name describing the exercise type>",
  "exercise_text": "<Instruction. If code is needed, embed it using a Markdown code block.>"
  "subtasks": {
    "subtask_1": "<First expression or question.>",
    "subtask_2": "<Second expression or question.>"
  }
}}
```

Figure 5: Structured prompt template for programming exercise generation. The purple highlights mark the dynamic context retrieved via RAG for each request (the learning objective and the semantically most similar lecture slides), while the remaining sections represent fixed components reused across all generations. The overall structure is inspired by [12].

5 Evaluation

In this section, we describe the semi-automated evaluation of the generated programming exercises. The goal is to assess their syntactic correctness, alignment with learning objectives, and consistency with Bloom's taxonomy levels. The evaluation is structured around the following research questions:

1. *RQ1: Are the generated programming exercises grammatically correct and readable for university-level students, and do the embedded Haskell code snippets compile without syntax errors?*
2. *RQ2: To what extent do the generated exercises make use of the contextual information provided in the related lecture slides?*

3. *RQ3: To what extent do the generated exercises align with the defined learning objectives and reflect the intended cognitive level according to Bloom’s taxonomy?*

We first introduce the selected models and experimental resources, followed by the automated evaluation pipeline. We conclude by presenting the results structured around the research questions.

5.1 Setup

The selected language model had to satisfy three requirements: (1) open-source availability for local deployment, (2) strong code understanding to interpret lecture slides, and (3) reliable code generation to create programming exercises.

We chose Qwen2.5-Coder [19], which performs on par or better than GPT-4o and DeepSeek Coder V2 on common coding benchmarks such as HumanEval, MBPP, and MCEval (see Table 1). To keep the setup reproducible on consumer hardware, we deployed the 7B variant locally on a MacBook Pro M2 Pro with 16 GB RAM. Even the 7B model achieves competitive results on standard benchmarks [19].

Benchmark	Qwen2.5-Coder 32B	GPT-4o	DeepSeek V2
HumanEval	92.7	92.1	88.4
MBPP	90.2	86.8	89.2
EvalPlus	86.3	84.4	83.8
MultiPLE	79.4	79.1	79.9
McEval	65.9	65.8	62.9
CRUXEval-O	84.2	89.2	75.1
BigCodeBench	38.3	37.6	36.3

Table 1: Coding benchmark comparison between Qwen2.5-Coder and commercial models [19].

For summarizing lecture slides and extracting key concepts, we used Gemma 3 (4B), a general-purpose open-source model developed by Google [35].

Decoding Parameters The Gemma 3 and Qwen2.5-Coder models were executed locally using Ollama, a lightweight framework to run LLMs on local machines [31]. For both models, we used the following decoding parameters: `temperature = 1.0`, `top_p = 1.0`, `frequency_penalty = 0.0`, and `presence_penalty = 0.0` (Ollama defaults). We increased `num_ctx` to 4096 to accommodate the combined input of slide summaries, learning objectives, and few-shot examples.

5.2 Experiment

We now describe the experimental resources used to generate programming exercises. As described earlier, the framework requires three inputs provided by the instructor: a topic, a learning objective, and lecture slides. In addition, it requires exercise sheets for each Bloom level to define the structure and style of the generated questions. The following paragraphs describe the input components and resources used in the experiment.

Learning Objectives and Topics To evaluate the framework across different levels of cognitive complexity, we created one example learning objective for each Bloom level. These were designed to represent meaningful and realistic learning objectives in the context of functional programming education. We generated ten exercises per learning objective, resulting in a total of 60 exercises for evaluation. The final set of topics and learning objectives used in our evaluation is summarized in Table 2.

Bloom Level	Topic	Learning Objective
Remember	Haskell Syntax and Prelude Overview	Students should be able to recognize and recall basic Haskell syntax, keywords, and built-in functions.
Understand	Type Inference and Haskell Type Signatures	Students should be able to describe the types of Haskell expressions and explain how type inference works using Haskell type signatures.
Apply	Rewriting Haskell Functions using Recursion	Students should be able to reimplement given Haskell expressions embedded in the task as equivalent recursive functions, without relying on higher-order functions.
Analyze	Analyzing Variants of Haskell Functions	Students should be able to analyze the structure of provided Haskell functions.
Evaluate	Functional Equivalence and Evaluation Behavior in Haskell	Students should be able to evaluate multiple Haskell implementations of the same function based on provided code examples, and justify which implementation is more appropriate under specific evaluation conditions.
Create	User-defined Data Types and Instances in Haskell	Students should be able to design custom data types and implement appropriate type class instances for them.

Table 2: Example Learning Objectives aligned with the six Bloom levels. These LOs serve as input to the exercise generation pipeline and form the basis for the subsequent evaluation.

Course Materials For the contextual input regarding lecture content, we used lecture slides from the Functional Programming course taught at TUHH by the Institute for Software Systems (STS Institute). The slides are real-world teaching materials that focus on Haskell programming concepts. In total, 504 PDF slides were provided to the framework as a complete set. From this corpus, we retrieved the top $k = 3$ slides per learning objective, balancing contextual coverage against prompt length: three slides typically capture the most relevant content while keeping the overall prompt compact.

Example Exercises To create example exercises, we selected exercises from the laboratory sessions of the Functional Programming course offered by the STS Institute. For each Bloom level, we identified three representative exercises from the lab sheets. These examples serve as templates to align the generated exercises with the expected style and difficulty per Bloom level.

Now that the experimental resources have been defined, the next section introduces the automated evaluation pipeline used to evaluate the quality of the generated exercises.

5.3 Automated Evaluation Pipeline

The goal was to evaluate the exercises generated with minimal manual effort. To achieve this, we identified a set of quality aspects that can be evaluated using automated methods. The pipeline consists of five stages, each designed to assess one specific criterion. In the following, we provide a detailed description of each stage.

Grammar The first stage verifies the grammatical correctness of the text in the generated exercises. Grammar analysis can be performed automatically using established tools or libraries. In our evaluation, we used `LanguageTool`, an open-source Python library for grammar analysis. `LanguageTool` applies a set of predefined rules to input text and reports errors when these rules are violated. We excluded several rules from the evaluation, as they incorrectly flagged code elements embedded in the exercises.

Examples include rules that expect sentences to begin with a capital letter (which is often not the case in code snippets) or rules that report double punctuation (e.g., expressions like `[1 . . 10]`). These rules were disabled to allow for Haskell code within the text while focusing on natural-language content. Note that these exclusions cover the most common cases but do not eliminate all possible false positives caused by embedded code.

Readability The second stage evaluates the readability of the generated exercise text using the Flesch Reading Ease Score [15], a widely used metric derived from average sentence length and average syllables per word. The score ranges from 0 to 100, where higher values indicate easier readability. Table 3 provides an interpretation of the score ranges.

Score Range	School Level
90–100	5th grade
70–90	6th to 7th grade
60–70	8th to 9th grade
50–60	10th to 12th grade
30–50	College
0–30	College graduate

Table 3: Interpretation of Flesch Reading Ease Scores [16].

Since the generated programming exercises are targeted at university students, an acceptable Flesch score lies between 0 and 50, corresponding to college-level and graduate-level difficulty.

Code Compilation The third stage evaluates the syntactic correctness of the code embedded in the generated exercises. Some exercises include code snippets that students are expected to analyze or answer questions about. To verify whether the generated code is syntactically correct, we perform an automated compilation check. We developed an extraction script that identifies all code segments embedded within the exercise text and writes them to a temporary Haskell file. This file is then executed using the Glasgow Haskell Compiler (`runghc`). If the code compiles without errors, the code within the exercise is considered syntactically correct. If the compilation fails, the corresponding exercise is flagged as syntactically incorrect.

Bloom Level Consistency The fourth stage evaluates whether the cognitive complexity of a generated exercise aligns with its intended Bloom level. For example, if an exercise was generated with the target level *Remember*, we would expect it to involve simple recall tasks. The key question is whether the final exercise actually reflects this intended level of complexity.

To assess this, we used open-source LLMs to automatically classify each generated exercise into one of the six Bloom levels. Each model received the full exercise text and was instructed to assign it to the most appropriate category using role prompting and detailed descriptions of Bloom’s taxonomy. For this classification, we employed two open-source models: Gemma 3 and Qwen2.5-Coder.

Lecture Slide Similarity The final stage measures the semantic similarity between the generated exercise and the relevant lecture slide summaries. Both the exercise and the summaries are first converted into vector representations. The cosine similarity is then calculated between the exercise vector and each slide summary vector. Higher similarity scores indicate that the exercise incorporates terminology and concepts from the lecture slides.

Each generated exercise passes through the entire evaluation pipeline. The pipeline stages map to the research questions as follows: Grammar, Readability, and Code Compilation address RQ1, Lecture Slide Similarity addresses RQ2, and Bloom Level Consistency addresses RQ3.

5.4 Results

The following subsections are structured by research questions. For each subsection, we first describe the aspects that were automatically evaluated by the pipeline, followed by the corresponding results. For aspects that could not be automatically evaluated, representative examples are included.

RQ1: Are the generated programming exercises grammatically correct and readable for university-level students, and do the embedded Haskell code snippets compile without syntax errors?

To answer RQ1, we applied the first three stages of the evaluation pipeline: grammar, readability, and code compilation. Table 4 summarizes the results.

Bloom Level	Grammar (%)	Flesch	Code Snippets	Compiles (%)
Remember	100	83.6	0	–
Understand	100	61.4	0	–
Apply	90	38.0	5	60
Analyze	90	42.3	5	100
Evaluate	90	32.6	8	50
Create	100	57.7	0	–

Table 4: Evaluation metrics for RQ1 per Bloom level: grammar correctness (% of exercises without LanguageTool issues), average Flesch readability score, number of exercises containing a code snippet, and compilation success rate on the generated snippets.

Grammar accuracy reached 100% for Remember, Understand, and Create, and 90% for Apply, Analyze, and Evaluate. The few detected issues in Apply, Analyze, and Evaluate were not genuine grammatical mistakes but residual false positives from code-related tokens that were not covered by our rule exclusions. Manual inspection confirmed that all flagged cases were artifacts of embedded code rather than actual grammatical errors.

Readability scores confirm that the exercises are appropriate for a university audience. *Apply*, *Analyze*, and *Evaluate* fall in the expected 30–50 range for college-level texts, reflecting their higher cognitive demands. In contrast, *Remember* (83.6) and *Understand* (61.4) scored higher, consistent with their simpler recall and descriptive character. Interestingly, *Create* reached 57.7, a higher readability score than one might expect given the reasoning required. This divergence is not a contradiction, as the Flesch score measures linguistic readability rather than cognitive difficulty.

Of the 17 exercises containing Haskell code, 12 passed the compilation check, resulting in an overall success rate of about 71%. Some failures occurred because the generated code was incomplete, even though the provided fragments were syntactically valid. For example, Listing 1 shows a case where a type signature was provided without a corresponding function body.

```
1 sumOfSquaresEven :: [Int] -> Int
2 main = return ()
```

Listing 1: Missing Definition for Declared Function.

Although the annotation itself was correct, the missing implementation caused the compiler to reject the code. Other compilation errors were genuine and reflected semantic issues. In one example, the standard function `filter` was applied to a custom-defined data type `List`, which lacks a compatible type-class instance. Since `filter` operates only on standard Haskell lists (`[a]`), the compiler correctly raised a type error.

Note that our compilation check implicitly assumes that the generated code is meant to compile. For debugging-style tasks with deliberately faulty code, a failed compilation would actually be the intended outcome. Since none of our 60 generated exercises contained such tasks, the compilation rate serves as an indicator of the model’s ability to produce syntactically correct Haskell code.

In summary, the results show that the generated exercises are grammatically correct and appropriately readable for a university-level audience. Nevertheless, 5 out of 17 code snippets (29%) failed to compile, highlighting an area where future work could improve robustness.

RQ2: To what extent do the generated exercises make use of the contextual information provided in the related lecture slides?

The second research question investigates whether the generated programming exercises leverage contextual information from the most relevant lecture slides. This aspect was evaluated in stage 5 of the pipeline, where embeddings of slide summaries and generated exercises were compared using cosine similarity. Additionally, we examine one representative example from the *Understand* level, which achieved the highest similarity score.

Bloom Level	Avg. Vector Similarity to Slides
Remember	0.506
Understand	0.607
Apply	0.445
Analyze	0.456
Evaluate	0.390
Create	0.498

Table 5: Average cosine similarity (range: -1 to 1) between each generated exercise and the summary of its associated lecture slides, aggregated by Bloom level. Higher values indicate stronger semantic alignment with the lecture content.

Table 5 summarizes the average semantic similarity between all 60 generated exercises and their corresponding lecture slide summaries. Cosine similarity ranges from -1 (opposite meaning) to 1 (identical meaning). Values around 0.4 – 0.6 indicate moderate semantic overlap, which is expected given that exercises and slides serve different purposes (teaching vs. assessment). The results show moderate similarity across all Bloom levels, with *Understand* reaching the highest value (0.607) and *Evaluate* the lowest (0.390). Extremely high similarity would not be desirable, as exercises and slides differ in structure and intent.

To illustrate the relationship between the generated exercises and the lecture slides, we now examine an example from the *Understand* level. The following lecture slide summaries provided contextual input to the generation process of the exercise shown in Figure 6.

1. *The slide introduces type classes in Haskell. A type class, `Eq`, defines operators like `(==)` and `(/=)` for equality and inequality checks. Many types such as `Int`, `Char`, and `Bool` are instances of `Eq`. Function types are not.*

2. *Types in Haskell are names for sets of related values. The `Bool` type denotes the values `True` and `False`. Variables and function arguments have types.*
3. *At the end of the course, students will be able to write Haskell programs, understand functional programming, and describe types and related concepts.*

These slides cover essential type-related concepts. Although the generated exercise does not reuse exact phrases, it clearly aligns with the discussed content. Students must reason about types (*summary 2*), equality (*summary 1*), and well-formed expressions (*summaries 1 and 2*). The third slide, describing general course goals, is less relevant to the concrete task.

Generated Exercise for the Understand Level

Which of the following Haskell expressions are well typed? Explain why each expression is either well-typed or not.

- a.) `head [404, 200]`
- b.) `length "Hello"`
- c.) `filter odd [1, 2, 3]`
- d.) `max (1, 2) (3, 4)`

Figure 6: Generated exercise targeting the Understand level of Bloom’s taxonomy.

We conclude that the generated exercises demonstrate moderate use of contextual information from lecture slides, as evidenced by similarity scores and qualitative analysis. While direct phrase reuse is rare, key concepts and terminology from the slides are reflected in the exercises.

RQ3: To what extent do the generated exercises align with the defined learning objectives and reflect the intended cognitive level according to Bloom’s taxonomy?

The third research question examines whether the generated exercises align with the defined learning objectives and whether their cognitive complexity matches the intended Bloom level. To evaluate this, we refer to stage 4 of the automated evaluation pipeline. In this stage, two open-source models were used to classify each generated exercise into one of the six Bloom levels. A total of 60 exercises was generated, 10 exercises per level. Table 6 summarizes the classification results.

Bloom Level	Gemma 3 (%)	Qwen2.5-Coder (%)
Remember	100	72.7
Understand	80	100
Apply	20	70
Analyze	70	70
Evaluate	53.3	93.3
Create	90	100

Table 6: Bloom level classification accuracy of Gemma 3 (4B) and Qwen2.5-Coder (32B) on the 60 generated exercises. Each value gives the percentage of exercises that the respective model classified as the intended Bloom level.

With the exception of *Remember*, Qwen2.5-Coder consistently achieved higher classification accuracy than Gemma 3. Gemma 3 showed particular difficulties at the *Apply* and *Evaluate* levels, both of which

frequently included embedded code snippets. One possible explanation is that Gemma 3 is a general-purpose model, whereas Qwen2.5-Coder is specifically optimized for programming-related tasks. Additionally, using Qwen2.5-Coder for both generation and classification may have led to more consistent cognitive framing, though this does not necessarily indicate better objective accuracy.

To further illustrate the observed classification behavior, we now examine a representative example from the *Evaluate* level. The programming exercise in Figure 7 was generated based on the learning objective: *Students should be able to evaluate multiple Haskell implementations of the same function and justify which implementation is more appropriate under specific evaluation conditions.*

Generated Exercise for the Evaluate Level

Consider the following two Haskell functions that compute the factorial of a number.

```

1 factorial1 n = product [1..n]
2 factorial2 0 = 1
3 factorial2 n = n * factorial2 (n - 1)
```

a.) Which implementation is more efficient for large input values? Explain your reasoning.
b.) Under what conditions might the second implementation be preferable despite its inefficiency?

Figure 7: Generated exercise targeting the Evaluate level of Bloom’s taxonomy.

The exercise aligns well with the intended learning objective, as students must compare two function implementations and justify their reasoning. However, Gemma 3 tended to classify similar tasks as *Analyze* rather than *Evaluate*, which is understandable given the conceptual overlap between comparing (analysis) and judging (evaluation) in Bloom’s taxonomy.

A similar pattern was observed for the *Apply* level, where Gemma 3 achieved only 20% accuracy. For several exercises at this level, students were required to rewrite given Haskell expressions as recursive versions. In these cases, Gemma 3 tended to classify such exercises as *Understand* rather than *Apply*, likely because comprehension of recursion is a prerequisite for its correct application.

Overall, most exercises were correctly classified by both models. The models performed particularly well at the Create and Understand levels, whereas lower scores at the Apply and Evaluate levels were likely due to conceptual overlap with adjacent Bloom levels. Despite classification discrepancies, manual inspection of selected exercises across Bloom levels confirmed that misclassified exercises often targeted the boundary areas between adjacent cognitive levels, suggesting that the generated exercises appropriately reflect the challenging transitional nature of these Bloom categories.

6 Limitations

During our evaluation, we observed some limitations that we would like to address here.

A first limitation concerns the scope of our evaluation: it is based on 60 exercises generated from a single Functional Programming course. Results may therefore not generalize to other programming paradigms, languages, or institutional contexts, and a broader study across courses and languages would be needed to assess the robustness of the approach.

A further limitation concerns the variation of the generated exercises. In a few cases, their structure closely mirrored the example exercises provided in the prompt, indicating that few-shot prompting can constrain variation. Domain diversity was also narrow at the *Create* level, where exercises often drew

from similar conceptual domains such as libraries, books, or students. Both effects may partly stem from the limited capacity of the 7B model used here.

Finally, the study lacks a systematic instructor evaluation. While the exercises were assessed automatically, pedagogical quality and classroom appropriateness were not validated by instructors or through deployment in an actual course.

7 Conclusion

This work introduced a framework that combines Bloom-classified learning objectives with retrieval-augmented lecture slide context to generate programming exercises using open-source LLMs. We evaluated the approach on 60 Haskell exercises generated with Qwen2.5-Coder (7B), using a semi-automated evaluation that combined an automated five-stage pipeline with manual review of selected examples. The generated tasks were grammatically correct, readable at university level, and aligned with the intended learning objectives, with 71% of embedded code compiling successfully. Building on these results, we outline several directions for future work.

On the technical side, exploring larger model variants such as 32B models is a logical next step beyond the 7B variant chosen here for accessibility, and could improve code robustness and output diversity. Refined prompt engineering with explicit diversity instructions may further reduce the structural similarity to few-shot examples observed in some cases, and incorporating richer slide representations such as diagrams or structured formats could enhance contextual grounding beyond plain-text summaries.

Beyond the technical pipeline, systematic instructor evaluations and an extension of the approach to multiple courses and programming languages would validate the pedagogical quality and generalizability of the framework.

References

- [1] Lorin W. Anderson & David R. Krathwohl (2001): *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. Longman.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye et al. (2021): *Program Synthesis with Large Language Models*. arXiv, doi:10.48550/arXiv.2108.07732. arXiv:2108.07732.
- [3] Rishi Bommasani, Jacob D. Hudson, Ehsan Adeli et al. (2021): *On the Opportunities and Risks of Foundation Models*. arXiv, doi:10.48550/arXiv.2108.07258.
- [4] Tom Brown, Benjamin Mann, Nick Ryder et al. (2020): *Language Models are Few-Shot Learners*. arXiv, doi:10.48550/arXiv.2005.14165.
- [5] Federico Cassano, John Gouwar, Daniel Nguyen et al. (2022): *MultiPL-E: A Scalable and Extensible Approach to Benchmarking Neural Code Generation*. arXiv, doi:10.48550/arXiv.2208.08227. arXiv:2208.08227.
- [6] Linzheng Chai, Shukai Liu, Jian Yang et al. (2024): *McEval: Massively Multilingual Code Evaluation*. arXiv, doi:10.48550/arXiv.2406.07436. arXiv:2406.07436.
- [7] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené et al. (2024): *Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review*. arXiv, doi:10.48550/arXiv.2310.14735.
- [8] Mark Chen, Jerry Tworek, Heewoo Jun et al. (2021): *Evaluating Large Language Models Trained on Code*. arXiv, doi:10.48550/arXiv.2107.03374. arXiv:2107.03374.

- [9] Shulin Chen & Qice Yan (2024): *A Chatbot for Learning and Life Guide in the University Based on LangChain+ChatGLM*. In: *2024 IEEE 4th International Conference on Software Engineering and Artificial Intelligence (SEAI)*, pp. 271–275, doi:10.1109/SEAI62072.2024.10674054.
- [10] Paul Denny, Juho Leinonen, James Prather et al. (2023): *Prompt Problems: A New Programming Exercise for the Generative AI Era*. *arXiv*, doi:10.48550/arXiv.2311.05943.
- [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee et al. (2019): *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. *arXiv*, doi:10.48550/arXiv.1810.04805. *arXiv*:1810.04805.
- [12] Jacob Doughty, Zipiao Wan, Anishka Bompelli et al. (2024): *A Comparative Study of AI-Generated (GPT-4) and Human-crafted MCQs in Programming Education*. In: *Proceedings of the 26th Australasian Computing Education Conference*, ACM, p. 114–123, doi:10.1145/3636243.3636256.
- [13] Matthijs Douze, Alexandr Guzhva, Chengqi Deng et al. (2025): *The Faiss library*. *arXiv*, doi:10.48550/arXiv.2401.08281. *arXiv*:arXiv:2401.08281.
- [14] Sabina Elkins, Ekaterina Kochmar, Jackie C. K. Cheung et al. (2024): *How Teachers Can Use Large Language Models and Bloom’s Taxonomy to Create Educational Quizzes*. *arXiv*, doi:10.48550/arXiv.2401.05914.
- [15] Rudolf Flesch (1948): *A New Readability Yardstick*. *Journal of Applied Psychology* 32(3), pp. 221–233, doi:10.1037/h0057532.
- [16] Rudolf Flesch (1979): *How to Write Plain English: A Book for Lawyers and Consumers*. Harper & Row.
- [17] Ian Goodfellow, Yoshua Bengio & Aaron Courville (2016): *Deep Learning*. MIT Press.
- [18] Marc Bessa Hoffmann (2025): *Automatically Generating Programming Exercises with Open-Source LLMs: Integrating Lecture Slides and Learning Objectives*. Research project, TU Hamburg.
- [19] Binyuan Hui, Jian Yang, Zeyu Cui et al. (2024): *Qwen2.5-Coder Technical Report*. *arXiv*, doi:10.48550/arXiv.2412.15115.
- [20] Yujia Jin, Guoyang Wu, Baoxing Yang et al. (2024): *LLM can Achieve Self-Regulation via Hyperparameter Aware Generation*. *arXiv*, doi:10.48550/arXiv.2402.11251. *arXiv*:2402.11251.
- [21] Takeshi Kojima, Shixiang Shane Gu, Machel Reid et al. (2023): *Large Language Models are Zero-Shot Reasoners*. *arXiv*, doi:10.48550/arXiv.2205.11916.
- [22] David R. Krathwohl (2002): *A Revision of Bloom’s Taxonomy: An Overview*. *Theory Into Practice* 41(4), pp. 212–218, doi:10.1207/s15430421tip4104_2.
- [23] Ghader Kurdi, Jared Leo, Bijan Parsia et al. (2020): *A Systematic Review of Automatic Question Generation for Educational Purposes*. *International Journal of Artificial Intelligence in Education* 30, pp. 121–204, doi:10.1007/s40593-019-00186-y.
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus et al. (2021): *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. *arXiv*, doi:10.48550/arXiv.2005.11401.
- [25] Subhankar Maity, Aniket Deroy & Sudeshna Sarkar (2025): *Leveraging In-Context Learning and Retrieval-Augmented Generation for Automatic Question Generation in Educational Domains*. *arXiv*, doi:10.48550/arXiv.2501.17397.
- [26] Christopher D Manning, Prabhakar Raghavan & Hinrich Schütze (2008): *Introduction to Information Retrieval*. Cambridge University Press.
- [27] Warren S McCulloch & Walter Pitts (1943): *A logical calculus of the ideas immanent in nervous activity*. *The Bulletin of Mathematical Biophysics* 5, pp. 115–133, doi:10.1007/BF02478259.
- [28] Niklas Meissner, Sandro Speth, Julian Kieslinger et al. (2024): *EvalQuiz – LLM-based Automated Generation of Self-Assessment Quizzes in Software Engineering Education*. In: *Proceedings of the Software Engineering und Unterricht 2024 (SEUH 2024)*, Gesellschaft für Informatik (GI), pp. 53–64, doi:10.18420/seuh2024_04. GI Lecture Notes in Informatics (LNI).

- [29] Sérgio Silva Mucciaccia, Thiago Meireles Paixão, Filipe Wall Mutz et al. (2025): *Automatic Multiple-Choice Question Generation and Evaluation Systems Based on LLM: A Study Case With University Resolutions*. In: *Proceedings of the 31st International Conference on Computational Linguistics*, Association for Computational Linguistics, Abu Dhabi, UAE, pp. 2246–2260. Available at <https://aclanthology.org/2025.coling-main.154/>.
- [30] NLTK Project: *Natural Language Toolkit (NLTK) Documentation*. <https://www.nltk.org>. Accessed: May 5, 2025.
- [31] Ollama: *Chat and Build with Open Models*. <https://ollama.com>. Accessed: May 5, 2025.
- [32] OpenAI: *Learn How to Prompt a Model to Generate Text*. <https://platform.openai.com/docs/guides/gpt>. Accessed: May 5, 2025.
- [33] Nils Reimers & Iryna Gurevych (2019): *Sentence-BERT: Sentence Embeddings using Siamese BERT Networks*. *arXiv*, doi:10.48550/arXiv.1908.10084. *arXiv*:1908.10084.
- [34] Nicy Scaria, Suma Dharani Chenna, Deepak Subramani et al. (2024): *Automated Educational Question Generation at Different Bloom’s Skill Levels Using Large Language Models: Strategies and Evaluation*, p. 165–179. Springer Nature Switzerland, doi:10.1007/978-3-031-64299-9_12.
- [35] Gemma Team, Aishwarya Kamath, Johan Ferret et al. (2025): *Gemma 3 Technical Report*. *arXiv*, doi:10.48550/arXiv.2503.19786. *arXiv*:2503.19786.
- [36] Hugo Touvron, Thibaut Lavril, Gautier Izacard et al. (2023): *LLaMA: Open and Efficient Foundation Language Models*. *arXiv*, doi:10.48550/arXiv.2302.13971.
- [37] University of Arkansas: *Using Bloom’s Taxonomy to Write Effective Learning Objectives*. <https://tips.uark.edu/using-blooms-taxonomy>. Accessed: May 5, 2025.
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar et al. (2023): *Attention Is All You Need*. *arXiv*, doi:10.48550/1706.03762. *arXiv*:1706.03762.
- [39] Shen Wang, Tianlong Xu, Hang Li et al. (2024): *Large Language Models for Education: A Survey and Outlook*. *arXiv*, doi:10.48550/arXiv.2403.18105. *arXiv*:arXiv:2403.18105.
- [40] Xuezhi Wang, Jason Wei, Dale Schuurmans et al. (2023): *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. *arXiv*, doi:10.48550/arXiv.2203.11171.
- [41] Jason Wei, Xuezhi Wang, Dale Schuurmans et al. (2023): *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. *arXiv*, doi:10.48550/arXiv.2201.11903.
- [42] Changrong Xiao, Sean Xin Xu, Kunpeng Zhang et al. (2023): *Evaluating Reading Comprehension Exercises Generated by LLMs: A Showcase of ChatGPT in Education Applications*. In: *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)*, Association for Computational Linguistics, Toronto, Canada, pp. 610–625, doi:10.18653/v1/2023.bea-1.52.
- [43] Lechen Zhang, Tolga Ergen, Lajanugen Logeswaran et al. (2024): *SPRIG: Improving Large Language Model Performance by System Prompt Optimization*. *arXiv*, doi:10.48550/arXiv.2410.14826.
- [44] Zixuan Zhang, Sidi Geng, Yutong Duan et al. (2024): *The Effect of Sampling Temperature on Problem Solving in Large Language Models*. *arXiv*, doi:10.48550/arXiv.2402.05201.
- [45] Zihao Zhou, Maizhen Ning, Qiufeng Wang et al. (2023): *Learning by Analogy: Diverse Questions Generation in Math Word Problem*. *arXiv*, doi:10.48550/arXiv.2306.09064.